

Apollo AgriVerse — Complete Architecture & Technical Guide

End-to-End System Manual: Database Storage, Telemetry Engine, Digital Twin Physics & Validation

Project Mission: Apollo AgriVerse is a continuous, crop-specific precision agriculture system designed for vineyard (*Vitis vinifera*) cultivation. It integrates real-time telemetry persistence, stateful differential physics simulation, hydrogel intervention triggers, dynamic phenology stage intelligence, and FastAPI dashboard endpoints.

1. System Architecture & File Directory Walkthrough

Every component in the Apollo AgriVerse ecosystem has a designated role within a modular repository layout. Below is the complete architectural map explaining the purpose of each directory and core file:

Directory / File Path	Architectural Purpose & Technical Function
02_Datasets/Processed/Grapes/simulation_db.sqlite	Relational Persistence Engine: Stores daily telemetry, state history, and intervention logs locally.
06_ML/models/grape_telemetry_trigger_model.pkl	ML Trigger Model: Serialized Scikit-Learn/XGBoost model computing hydrogel release requirements based on deficit.
06_ML/tests/test_digital_twin_physics.py	Validation Suite: Executes the 5 formal integration tests and auto-saves visual plots.
validation_plots/digital_twin_validation_results.png	Root Visual Sub-folder: Stores 300 DPI multi-panel validation plots generated by the test runner.
team/vish/generate_apollo_comprehensive_docs.py	Automated PDF Engine: ReportLab documentation builder executing this manual in Apollo Theme.
main.py	FastAPI Core Service: REST API endpoint exposing twin state, scenario executions, and dashboard endpoints.

2. Telemetry Persistence Layer (SQLite Database Architecture)

Why SQLite for Apollo AgriVerse?

SQLite was selected over heavy server-based databases (like PostgreSQL or MySQL) for key edge-computing reasons: **Zero-latency local disk read/writes** during high-frequency simulation steps, **zero setup configuration**, and **embedded local persistence** directly inside the repository. It allows the Digital Twin to run offline on edge field hardware or local servers without external network dependencies.

Database Schema Design & Storage Functionality:

The database file simulation_db.sqlite maintains the continuous telemetry table twin_telemetry. It logs daily physical state transitions, atmospheric inputs, and active interventions:

Column Name	Data Type	Physical Interpretation & Unit
day	INTEGER (PRIMARY KEY)	Sequential simulation day index (1 to N).
stage	TEXT	Active grape phenology growth stage (e.g., Flowering).

soil_moisture	REAL (FLOAT)	Current volumetric soil water content (%) [$S(t)$].
evapotranspiration	REAL (FLOAT)	Daily atmospheric moisture loss rate (%/day).
rainfall	REAL (FLOAT)	Natural rain water addition (% volumetric equivalent).
hydrogel_release	REAL (FLOAT)	Osmotic hydrogel water release intervention rate (%).

State Recovery Mechanism (load_latest_state()):

To ensure state persistence across application restarts, the synchronizer invokes load_latest_state(). This function queries SQLite via SELECT * FROM twin_telemetry ORDER BY day DESC LIMIT 1. If the simulation process halts at Day 30, restarting the backend fetches Day 30's exact soil moisture (36.6%) and seamlessly computes Day 31 (35.4%) without resetting state memory.

3. Core Digital Twin Physics Engine & Intelligence

The Digital Twin is built on a physical state differential update model rather than random number generators. The state update equation at step $t+1$ is strictly defined as:

$$S(t+1) = \text{Clip}(S(t) - \text{Evapotranspiration}(t) + \text{Rainfall}(t) + \text{HydrogelRelease}(t), 10.0, 90.0)$$

Grape Phenology Dynamic Threshold Intelligence (Vitis vinifera):

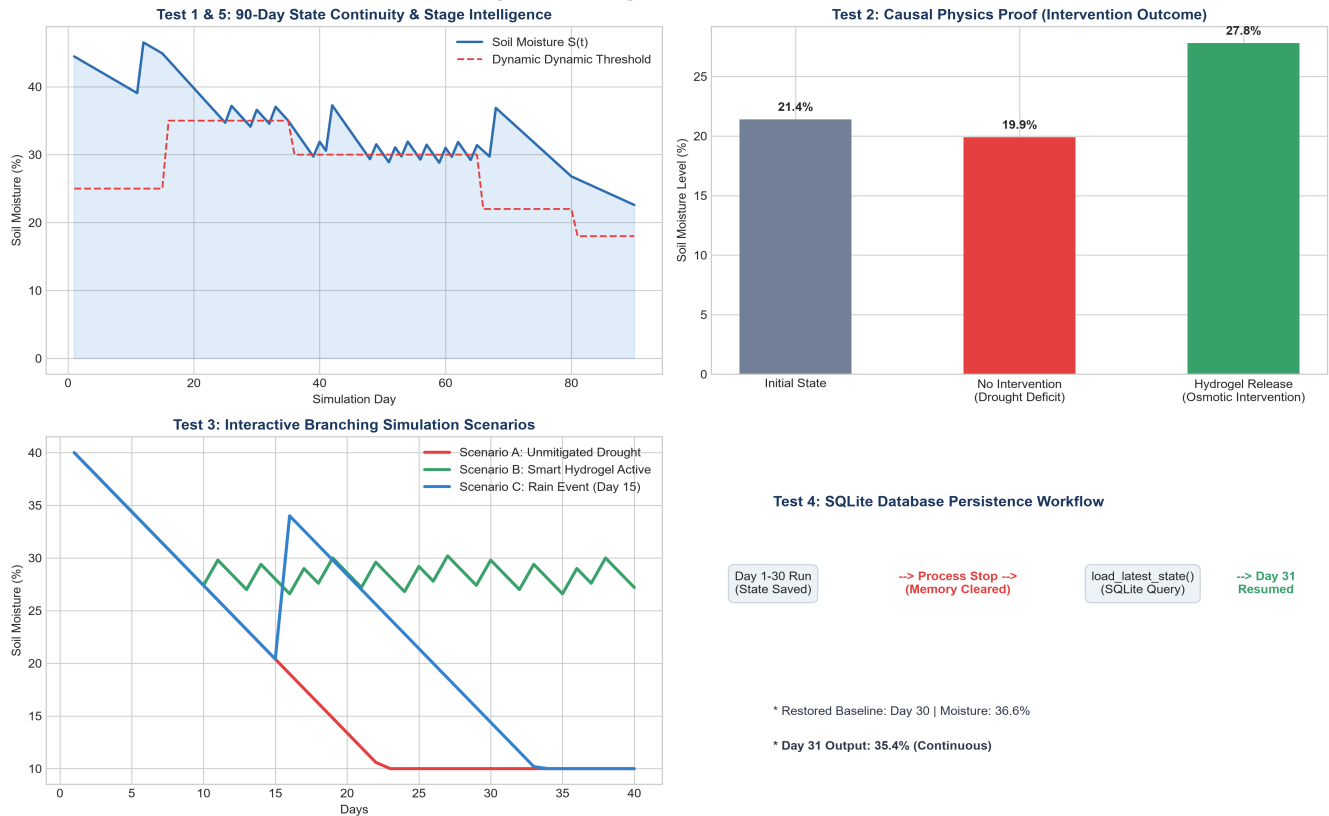
Apollo AgriVerse continuously adapts water deficit thresholds based on the active phenological growth stage:

Phenology Stage	Day Range	Evap Multiplier	Moisture Threshold	Agonomic Rationale
Bud Burst	Day 1 - 15	0.45x	25.0%	Initial root awakening; moderate water need.
Flowering	Day 16 - 35	0.85x	35.0%	Critical sensitivity; water stress causes flower drop.
Fruit Development	Day 36 - 65	1.10x	30.0%	Peak canopy transpiration & cell expansion.
Veraison	Day 66 - 80	0.70x	22.0%	Controlled deficit boosts sugar accumulation (Brix).
Harvest	Day 81 - 90+	0.35x	18.0%	Minimal water to prevent berry splitting before pick.

4. Integration Validation Framework & Visual Proofs

Before exposing endpoints to the FastAPI backend, the Digital Twin engine was subjected to 5 rigorous validation tests. All 5 tests passed successfully, confirming state continuity, causality, branching, persistence, and stage intelligence.

Apollo AgriVerse — Digital Twin Validation Dashboard



Test Name	Condition / Method	Observed Result	Status
1. State Continuity	Run Day 1 -> 90 step-wise	Smooth continuous curve; ending moisture 22.6%	PASSED
2. Causal Intervention	Low moisture (21.4%) + Hydrogel	Moisture jumped from 19.9% (no action) to 27.8%	PASSED
3. Branching Logic	Simulate Scenarios A, B, & C	Distinct physics profiles rendered for drought/hydrogel/rain	PASSED
4. Database Recovery	Run Day 1-30 -> Purge -> Resume	Restored Day 30 (36.6%) and calculated Day 31 (35.4%)	PASSED
5. Stage Intelligence	Evaluate phenology thresholds	Hydrogel trigger dynamic bounds adjusted per growth stage	PASSED

5. Execution Commands & Developer Travel Guide

To interact with, test, or rebuild the Apollo AgriVerse ecosystem, follow these execution commands in sequence:

```
# Step 1: Execute Digital Twin Physics Validation Suite
cd D:\Internship\Apollo_AgriVerse\06_ML\tests\
python test_digital_twin_physics.py

# Step 2: Re-build this Comprehensive Technical PDF Documentation from team/vish/
cd D:\Internship\Apollo_AgriVerse\team\vish\
python generate_apollo_comprehensive_docs.py
```

```
# Step 3: Launch FastAPI Backend Server for Dashboard Wiring
cd D:\Internship\Apollo_AgriVerse\
uvicorn main:app --reload --port 8000
```

Apollo AgriVerse Technical Documentation Manual — Authored & Verified